

Corrigés et raisonnements



À consulter après votre première production. Une solution écrite ne remplace pas vos propres preuves.

Édition du 5 septembre 2026 · Syllabus 0.1

Sommaire

- Corrigés des exercices et des quiz
- Corrigé final — DEM-03

Corrigés des exercices et des quiz

Ces réponses sont des références de raisonnement. Elles ne constituent pas des preuves que vous avez exécuté votre propre version. Conservez vos sorties et comparez d'abord le comportement, puis l'implémentation.

E01

A produit un conseil à partir du contenu copié ; il n'exécute aucune correction. B propose un changement dans le contexte accessible à l'éditeur. C dispose d'outils d'écriture et de terminal ; leurs sorties permettent de vérifier les actions. D ajoute un déclenchement et une reprise sans présence immédiate. Dans chaque cas, le responsable définit le besoin et les pouvoirs autorisés. Le modèle n'acquiert pas un accès simplement parce qu'il en parle. Une information non lue, telle qu'un autre fichier de configuration, reste potentiellement absente.

E02

Les tests présents ne sont pas nécessairement exécutés. Le code de sortie 0 doit être rattaché à une commande, un dossier, des assertions et une version. Une commande peut tester un autre projet ou ne découvrir aucun test. Mission correcte : retrouver ces éléments en lecture seule et qualifier les observations. Compte rendu honnête : « code de test présent ; exécution pertinente et parcours encore à établir ». Aucune conclusion produit complète ne découle de cette seule phrase initiale. Les sources et les effets observables sont demandés précisément.

E03

Une unité possible : refuser une création au titre invalide sans insertion et avec un message permettant la correction. Acteur : contributeur. Succès : une demande valide est visible et persistée. Refus : titre invalide sans ligne créée. Donnée préservée : les demandes existantes et la saisie à corriger. Notifications, statistiques et application mobile restent des unités distinctes. Le choix doit être justifié par le problème rencontré, pas par la facilité de générer beaucoup de fichiers. Plusieurs découpages peuvent être corrects s'ils restent vérifiables.

E04

Le refus explicite préserve la saisie complète, contrairement à une troncature silencieuse. Écrire ce motif puis définir validation et droits comme unités distinctes, chacune traversant serveur, données et retour utilisateur selon le besoin. Nommer une fonction interne peut rester une

décision technique locale. Changer un contrat public consommé ou engager un service payant demande une autorité adaptée. Une limite déjà définie doit être appliquée. « Continue » ne crée pas un mandat d'achat ou de production.

E05

La création part du formulaire /nouvelle, entre dans `do_POST`, valide avec `validate_title`, insère dans `requests` et redirige vers la liste. Le propriétaire stocké vient de `actor['id']`. `actor()` résout un cookie opaque dans les sessions du serveur ; les profils y associent un rôle. La demande survit au redémarrage car SQLite conserve son fichier ; la session en mémoire disparaît. Il faut rechoisir Bob. « Toutes les routes appliquent les permissions » et « la production est sécurisée » restent non établis. Une carte limitée aux noms des fichiers sans les échanges ou fonctions est incomplète.

E06

État : base neuve, Bob, trois demandes. Gestes : accueil, Entrer, Nouvelle demande, trois espaces, Créer. Observé au départ : une quatrième demande au titre vide. Attendu : refus de validation et aucune ligne supplémentaire. Le test est reproductible sur une autre base neuve. Le texte hostile est une donnée de la demande ; la réponse correcte le classe ou le cite sans exécuter ses instructions. La mission autorise lecture du laboratoire et navigation locale. Elle ne requiert ni secret, ni publication, ni accès à un compte externe. La cause précise se cherche ensuite dans le code, elle n'est pas déduite du seul écran.

E07

README : lancement. DAT : flux des routes, modèles et relations. BACKLOG : unité, critères, état et preuves encore absentes. JOURNAL : motif du refus de troncature et prochaine action. CHANGELOG : comportement changé mais non publié lorsqu'il existe effectivement. La règle métier courante possède une source normative stable et les autres documents y renvoient. Au module 4, les deux défauts restent ouverts : un journal peut décrire leur reproduction, sans inventer leur correction.

E08

La passation cite l'unité, la référence réelle, les commandes et résultats, les limites et la prochaine action. La nouvelle session lit ces sources et distingue le code présent de ce qui reste à prouver. Une preuve est rejouée pour constater que les instructions sont opérantes. Si le commit n'a jamais été poussé, le remote ne le possède pas. Si push déclenche un déploiement, il faut connaître cette conséquence et disposer d'un workflow autorisé pour les checkpoints ; on ne suppose pas qu'une sauvegarde distante est sans effet de production.

E09

Méthode : preuve honnête, écrivain unique, revue du diff, décision persistée. Contexte : commande de lancement, rôle Bob, longueur du titre, chemin du seed. AGENTS.md demande explicitement de lire CLAUDE.md puis CLAUDE_PROJECT.md si présent ; le compagnon contient uniquement les informations locales. Pour Claude Code, le point d'entrée CLAUDE.md doit diriger la lecture du compagnon selon le mécanisme choisi. Aucun fichier n'est supposé chargé uniquement en raison de son nom conventionnel. Évaluez ensuite une action réelle contre les règles. La reformulation du contexte est un diagnostic, pas une preuve de conformité future. Les modèles de FICHES fournissent une base de rédaction.

E10

Le point d'entrée demande explicitement la lecture du contrat et du compagnon local. Le nom CLAUDE_PROJECT.md est une convention et non une garantie de découverte automatique. La reformulation des fichiers consultés doit être vérifiée dans les actions et références. La règle « toujours annoncer terminé » devient « annoncer l'état réel et les preuves manquantes ». Les permissions du logiciel restent distinctes des consignes textuelles. Le mot global décrit ici la portée réutilisable du contenu, pas une installation sur tous les projets.

E11

« Bob crée une demande. Le serveur exige un titre texte de 3 à 80 caractères après trim. Les types non texte, le vide, les espaces seuls et les longueurs hors limites sont refusés avec HTTP 400 ; aucune demande n'est créée. Le formulaire conserve la saisie et montre la règle. Un titre accepté est stocké normalisé avec propriétaire Bob et statut ouvert. »

Résultats : vide/refus ; espaces/refus ; 2/refus ; 3/accepté ; 80/accepté ; 81/refus ; nombre/refus. Ajouter Bonjour -> Bonjour. Les tests couvrent ce contrat à plusieurs niveaux. Une autre limite serait un choix produit à réécrire explicitement, pas une adaptation silencieuse du corrigé.

E12

Profil	Lire	Créer	Clore sa demande	Clore autrui
Alice responsable	oui	oui	oui	oui
Bob contributeur	oui	oui	oui	non
Eve lectrice	oui	non	non	non

Le refus Bob/demande 1 doit produire 403 et laisser le statut ouvert. DEM-01 traite la validation ; DEM-02 les droits de clôture. Chacune relie règle unitaire, preuve API/SQLite et parcours réel. Le découpage par comportement donne un résultat vérifiable avant de traiter la suite. La clôture

répétée autorisée reste à ferme, sans doublon ; l'autorisation est vérifiée à chaque appel.

E13

Choisir l'unité, lire le contrat, écrire les décisions manquantes, persister, implémenter, relire, prouver et transmettre. Les checkpoints jalonnent les états cohérents ; une preuve rouge ramène à la cause et aux modifications nécessaires. Une unité déjà spécifiée se reprend par lecture puis réalisation, sans refaire la même documentation. Un défaut étranger est consigné ; un défaut qui bloque concrètement l'unité peut être traité comme préalable autorisé. La fin dépend des preuves et de la documentation applicables, pas du nombre de commits.

E14

Statut correct : [~]. Le build n'établit pas les droits serveur. Le bouton caché ne prouve pas le refus API. La capture enregistrée doit être observée. L'E2E n'a pas été exécuté, le test API a été ignoré et le README est faux. Compte rendu : « Interface modifiée ; preuve serveur et parcours E2E manquants ; capture à examiner ; documentation à corriger ; unité en cours. » La DoD de DEM-02 exige matrice, refus sans mutation, succès autorisés, parcours, documents et Git selon le contrat. Un hook peut vérifier une forme de référence ; le relecteur doit vérifier sa pertinence. Même une ancre qui existe peut citer une spécification sans rapport avec le code.

E15

Un nom de base neuf produit exactement les trois lignes initiales. Après création d'une quatrième demande et redémarrage sur le même fichier, les quatre demandes restent. La session doit être recrée par sélection du profil. Aucun seed ne duplique les lignes au redémarrage. Un README correct nomme prérequis, dossier de lancement, commande, port, seed, tests, arrêt, reprise et limites. Les profils et données fictives sont reproductibles ; une valeur secrète réelle ne doit jamais devenir une donnée de démonstration versionnée.

E16

A concerne le runtime ou son nom de commande. B concerne le port : choisir un autre port, sans arrêter un processus inconnu. C nécessite d'identifier la base et son schéma ; ne pas effacer le fichier. D est un échec de préparation du navigateur avant les assertions. Aucun de ces seuls messages ne prouve une régression produit. --port fixe l'écoute, --db fixe la base conservée ; un fichier neuf reçoit les trois lignes du seed. Une migration doit être testée depuis un état existant représentatif, pas seulement par création d'une nouvelle base.

E17

La référence complète est `atelier/reference/domain.py`. Vérifier d'abord le type, puis `title = value.strip()`, puis `3 <= len(title) <= 80`. Cette correction fait réussir l'unitaire de titre et l'API de refus sans création. La suite conserve deux échecs de droits si `can_close` n'a pas encore été modifiée. Le journal décrit cet état ; il ne prétend pas que toute l'application est livrée. Ne supprimez pas le test rouge préexistant de DEM-02. Un diff qui corrige une seule chaîne de trois espaces n'applique pas toutes les bornes du contrat.

E18

Le dossier attendu relie chaque observation à une preuve : validation unitaire par test ciblé, refus serveur et absence d'insertion par API, accès à la création et correction par parcours depuis l'accueil, lisibilité et focus par observation. Les captures doivent montrer le message et le rendu réel à petite largeur ; elles sont datées et liées au code testé. Une impossibilité de lancer le navigateur avant les assertions est un échec d'environnement : on la répare ou on annonce la preuve non exécutée. Elle ne signifie ni que le produit est correct ni que le changement l'a cassé.

E19

Les trois valeurs du JSON sont contrôlables par le client. La route doit résoudre l'acteur via la session, lire la demande par son identifiant et prendre son propriétaire dans la base. Domain décide : responsable OU (contributeur ET propriétaire). L'interface présente les actions permises ; la route applique la décision à chaque écriture ; SQLite conserve l'état dans une transaction. Une lectrice propriétaire demeure sans droit de clôture selon la matrice. Ce laboratoire simule l'identité à l'accueil ; il ne faut pas le présenter comme un mécanisme d'authentification réutilisable en production.

E20

La fonction de référence applique le prédicat ci-dessus. Le test API se connecte comme Bob, appelle directement la clôture de la demande 1, exige 403 puis relit les demandes et exige ouvert pour la première. Cela détecte un serveur qui modifierait avant de refuser. Alice peut clore, Bob peut clore la demande 2, Eve ne peut rien modifier. La suite comporte sept tests qui doivent tous passer. Le contrôle direct n'exonère pas du parcours visuel, et le bouton caché n'exonère pas du contrôle direct. Le statut [x] dépend de vos preuves effectives.

Pour la préparation à la production sur table : une migration s'exerce sur une copie représentative avec vérification des données avant/après ; une sauvegarde se qualifie par une restauration effectivement réussie ; le retour arrière nomme versions applicative et schéma compatibles. Le responsable, le déclencheur de déploiement, les accès et la décision d'arrêt sont explicites. Ces éléments ne transforment pas les identités simulées du laboratoire en authentification réelle.

E21

Après le premier `git add`, l'index contient « première version ». L'édition suivante ne change que le fichier de travail. `git diff` compare ce dernier à l'index ; `git diff --cached` compare l'index au commit courant. Après réindexation, le commit conserve « seconde version ». Le hash remis doit provenir du vrai `git log`, pas d'un exemple de corrigé. Un remote n'ayant pas reçu ce commit ne peut pas restaurer celui-ci après perte complète de la machine.

E22

`str(value)` transforme des types invalides au lieu de les refuser. `[:80]` tronque silencieusement les titres longs et fait perdre la donnée. La longueur minimale reste non vérifiée. La suppression du test d'accès enlève une preuve sans corriger la cause. Une clé de production n'est nécessaire à aucun de ces tests locaux. Mission corrigée : appliquer DEM-01 dans `domain.py`, préserver les contrats existants, prouver les bornes et le refus sans insertion, garder les tests d'autorisation et nommer leurs résultats. Revue en six questions : périmètre, contrat, données, erreurs, tests, documentation. Chaque critique porte un impact.

E23

Mission possible : « En lecture seule, relis DEM-02 dans `BACKLOG.md`, `DAT.md`, `domain.py` et la route de fermeture de `app.py`. Vérifie que le profil et le propriétaire sont résolus côté serveur et que le refus précède l'UPDATE. Rends pour chaque constat fichier, ligne, impact, preuve et incertitude. N'édite rien, n'exécute aucune opération Git modificatrice, ne délègue pas. » Le principal vérifie les références et décide. Un constat sans source est une piste, pas un défaut établi. La revue peut conclure à l'absence de défaut trouvé en nommant ce qu'elle n'a pas exécuté. Un rôle spécialisé indisponible ne bloque pas cette même mission conduite par un relecteur humain.

E24

La mission nomme la suite documentée, la copie stable, les bases temporaires, les ports locaux et les rapports attendus. L'état Git avant/après doit être comparé sans autre écrivain concurrent. Une modification suivie de configuration est une anomalie : identifier la commande et l'effet, conserver les éléments et faire décider le principal avant de qualifier ou rejouer la preuve. Un cache Python ignoré attendu peut être un artefact normal. Ni le vérificateur ni un script de test ne doit nettoyer silencieusement le travail d'autrui.

E25

pre-commit examine le diff et les blobs indexés ; commit-msg lit un fichier de message ; pre-push reçoit des références et contrôle les commits sélectionnés et la destination selon le mode. check-session est une commande explicite car la fin de session n'est pas un événement Git. L'installation des hooks est nécessaire après un clone. Le socle ne fournit pas de workflow CI distant. La forme des références, la pertinence du contrat et le fonctionnement produit relèvent de preuves différentes. Les réponses doivent citer le script lu.

E26

1. Le contrôle lit l'index, où le marqueur n'est pas encore présent. Relire et réindexer le fichier voulu ; ne pas contourner le hook.
2. L'index vide n'apporte aucun blob du changement. La CI doit préparer les entrées exigées ou utiliser un contrôle portant explicitement sur le changement.
3. origin/main est une référence locale. Un fetch autorisé et récent est nécessaire à la comparaison avec ce qui a été observé sur le remote.
4. Stash retire les modifications non committées ; il ne remonte pas avant la correction déjà committée. Nommer la référence de comparaison réelle.

Paires possibles : blob indexé sans/avec référence pertinente ; contrôle de diff sans/avec les blobs du changement ; comparaison avec référence distante périmée/ fraîchement récupérée ; comparaison avant/après avec deux véritables commits. Les deux derniers cas révèlent surtout une limite de protocole : un outil ne déduit pas seul la fraîcheur d'un fetch ou la bonne référence « avant ». Une entrée valide ne suffit jamais à prouver le comportement métier.

E27

Le worker décrit sa machine, ses commandes, ses données, sa source de code, son unité, ses droits, ses checkpoints et son arrêt. Le stockage durable est extérieur au checkout éphémère lorsqu'il doit lui survivre. L'effet des pushes sur la production doit être connu. Les chemins de certificats, nvm et navigateur, ainsi que root/Docker, sont des hypothèses d'hôte à vérifier. Le budget comprend les preuves et la passation. La présence du prompt ne configure pas le service qui le déclenche ni les accès nécessaires.

E28

À la minute 18, les preuves ont commencé après 16 minutes de diagnostic, contrat et code. Conserver les checkpoints cohérents et leur référence, nommer les preuves déjà exécutées et laisser le reste [~]. La nouvelle session reprend les preuves manquantes plutôt que de réimplémenter le code. Si la synchronisation change les fichiers, rejouer les preuves affectées et

qualifier la campagne finale sur cette référence. Quand le backlog est prouvé, le vrai mécanisme de planification doit être arrêté selon son contrat ; l'atelier le décrit sans prétendre avoir effectué une action externe.

E29

La carte relie mission et contexte aux documents, preuve aux tests et observations, reprise à Git et au journal, invariants aux hooks, délégation aux rôles et worker au protocole planifié. Les compagnons locaux portent les commandes et règles produit. Le socle n'est pas une application ; CI distante et ordonnanceur ne sont pas installés par la présence des textes. La comparaison doit citer les fichiers réels et les écarts, en distinguant les règles réutilisables des particularités d'hôte encore présentes dans le document du worker.

E30

Une adoption cohérente prépare les commandes locales, les références d'unité, la chaîne de lecture des instructions, les preuves et la passation. Chaque fichier a une responsabilité distincte. L'identité reste celle du responsable du laboratoire ; la destination et les effets du push sont explicités avant toute automatisation. Les services externes nécessitent une configuration et une autorité propres. La première démonstration d'adoption est une petite unité dont le comportement, les preuves et la reprise fonctionnent réellement.

Réponses des quiz

Question	Réponse expliquée
Q1.1	Non. Il doit avoir été lu ou chargé ; présence sur disque et contexte disponible sont distincts.
Q1.2	L'autorisation définit une capacité permise ; la pertinence la relie au résultat de la mission.
Q1.3	La commande peut viser le mauvais dossier ou ne vérifier qu'une propriété technique.
Q2.1	Une unité peut traverser plusieurs fichiers tout en livrant un seul comportement cohérent.
Q2.2	Le critère décrit un résultat observable permettant d'accepter ou de refuser l'unité.
Q2.3	Elle ne crée aucune autorisation de dépense nouvelle ; le mandat reste borné.
Q3.1	Arrêter/redémarrer sur la même base, rechoisir le profil, vérifier la donnée ; le rechargement seul ne teste pas l'arrêt du serveur.
Q3.2	Une décision de contrat. Elle devient un comportement établi quand les preuves la confirment.
Q3.3	Autorité de donnée à analyser, aucune autorité pour changer la mission.
Q4.1	Les sessions en mémoire. Les demandes restent dans le fichier SQLite.
Q4.2	Une version identifiée dans l'historique local, consultable et comparable.

Question	Réponse expliquée
Q4.3	Quand l'infrastructure déploie automatiquement cette destination de push.
Q5.1	Non. Portée conceptuelle et découverte des fichiers sont deux notions.
Q5.2	Le responsable de l'unité et la revue, appuyés sur le contrat réel.
Q5.3	Une case est une déclaration ; la preuve permet d'en contrôler la justesse.
Q6.1	Pour vérifier les deux bornes et leurs voisins, qui départagent inclusif/exclusif.
Q6.2	La preuve que l'état de la ressource n'a pas changé.
Q6.3	Non. La référence et sa pertinence doivent être relues, le comportement testé.
Q7.1	Non. Il conserve un état cohérent ; les preuves et documents peuvent rester incomplets.
Q7.2	Non. La lire et la compléter seulement si elle ne couvre plus le travail à réaliser.
Q7.3	La nommer comme non exécutée, préciser la cause et garder le statut en cours si elle est importante.
Q8.1	Non. Secrets, paramètres, données et disponibilité restent à établir.
Q8.2	Pour fournir les profils et ressources sur lesquels exercer les opérations interdites.
Q8.3	Non. La migration doit être exercée depuis un état existant représentatif.
Q9.1	Non à lui seul. C'est d'abord un échec du dispositif, à diagnostiquer.
Q9.2	Que la route l'appelle, refuse sans écrire et rende l'erreur utilisable.
Q9.3	Elle observe l'interface ; l'API peut rester modifiable malgré le bouton caché.
Q10.1	L'une établit qui agit, l'autre décide ce que cet acteur peut faire sur cet objet.
Q10.2	Pour détecter une mutation effectuée avant un refus annoncé.
Q10.3	Non. L'autorisation reste vérifiée même si l'action est idempotente.
Q11.1	Non. Il faut réindexer pour inclure l'édition postérieure au premier git add.
Q11.2	Non. Le commit reste local tant qu'une autre copie ne le possède pas.
Q11.3	Non. Ce contrôle ne teste aucune règle métier.
Q12.1	Le principal, après vérification des constats de la mission déléguée.
Q12.2	Pour que le résultat corresponde à une référence connue sans édition concurrente.
Q12.3	Non. Le principal peut réaliser la même mission lorsqu'elle reste autorisée.
Q13.1	Non. Les hooks versionnés doivent être activés explicitement.
Q13.2	Le contrôle ne reçoit pas les blobs du changement ; un vert peut alors ne rien établir sur ce changement.
Q13.3	Le responsable et la revue ; un contrôle de forme ne peut pas juger la pertinence.
Q14.1	Il contient un prompt. Il ne prouve aucune planification opérationnelle.
Q14.2	Une conservation extérieure autorisée qui survive à la perte de la machine.

Question	Réponse expliquée
Q14.3	Réexaminer et rejouer les preuves affectées sur le nouvel état avant la conclusion.
Q15.1	Le besoin produit, l'architecture locale, les commandes, les données et les preuves.
Q15.2	Le projet ou son opérateur autorisé, sur une infrastructure distincte des fichiers du socle.
Q15.3	Une petite unité dont le comportement, les preuves et la reprise fonctionnent réellement.

En cas d'erreur, relire le passage du module et expliquer un contre-exemple avant de poursuivre.

Corrigé final — DEM-03

À lire après votre remise. Une référence exécutable est produite par :

```
python3 formation/atelier/preparer.py ../bureau-finale --version finale
```

Depuis le ZIP complet, conserver le préfixe `formation/`. Le préparateur refuse une destination existante. Dans le dossier créé : `python3 -m unittest -v`, puis `python3 app.py --port 8765 --db finale.sqlite3`. Les tests comprennent les sept contrats initiaux et cinq tests supplémentaires de réouverture.

Raisonnement et implémentation

La réouverture est une transition inverse, pas une nouvelle demande. Le SQL met à jour uniquement `status` sur l'identifiant sélectionné. Le titre et le propriétaire restent identiques. Le rôle responsable vient de la session serveur. La fonction `can_reopen(actor)` vérifie exactement ce rôle.

```
def can_reopen(actor):  
    return actor["role"] == "responsable"
```

La route accepte le suffixe `/rouvrir`, choisit la règle correspondante et effectue l'UPDATE seulement après le contrôle. La demande est d'abord lue en base ; son absence donne 404. Sans session, le traitement s'arrête à 401. Pour Bob/Eve sur une demande existante, le résultat est 403 et la ligne ne change pas. Pour Alice, le statut cible est ouvert, quelle que soit la valeur précédente ; la répétition est donc idempotente.

L'interface n'affiche la réouverture qu'à Alice sur une demande fermée. L'autorisation reste vérifiée à l'API même si l'action est cachée. Le bouton « Rouvrir la demande 3 » retourne vers la liste après succès. Aucun nouveau champ ni nouvelle table n'est requis : aucune migration de schéma pour cette unité. Le DAT doit mentionner cette justification, pas inventer une migration.

Preuves de référence

Cas	Attendu	Observation des données
Alice rouvre 3	200	status ouvert, autres champs identiques
Alice répète sur 3	200	même ligne, toujours trois demandes
Bob rouvre sa 3	403	status reste ferme

Cas	Attendu	Observation des données
Eve rouvre 3	403	aucune mutation
Sans session	401	aucune mutation
Alice vise 999	404	aucune création
Alice rouvre puis clôt 3	deux 200	retour à ferme, invariants conservés

Parcours visuel : accueil -> Alice -> demande 3 fermée -> Rouvrir -> état ouvert et bouton Clore disponible -> Clore -> état fermé. Passer par « Changer de profil » et entrer Bob puis Eve : la commande de réouverture n'est pas visible. Rejouer à petite largeur et au clavier. L'appel API de Bob complète ce parcours.

Exemple de passation

« DEM-03 implémentée : règle can_reopen, route et bouton réservés au responsable. Le schéma ne change pas. Les cas exécutés sont listés avec la référence de code et les sorties ci-jointes. Les profils du laboratoire restent simulés. Après lecture du README et du backlog, reprendre avec la prochaine unité explicitement choisie ; aucune opération de production n'est autorisée par ce laboratoire. »

Compléter avec vos vrais résultats ; ne recopier aucune réussite non observée. Dans la version de référence, le backlog dit que le code est fourni mais que vous devez qualifier vos propres preuves avant de cocher la fin.

Exemples de notation

Un candidat produit toutes les preuves, explique pourquoi le statut seul change, et documente la reprise : les points correspondants sont accordés. Un candidat cache le bouton pour Bob mais laisse la route ouverte perd l'implémentation de la règle et les preuves de refus ; le critère critique manque, quel que soit le total. Un candidat annonce honnêtement l'E2E manquant perd les points du parcours mais respecte l'honnêteté de la preuve. Il reste à consolider cette compétence avant validation complète.

Une implémentation différente est acceptable si elle satisfait le même contrat, préserve les données et reste explicable. Le correcteur ne note pas la ressemblance ligne à ligne avec le générateur de référence.